Fundamentals of Testing

What is the scope of software testing?

How does testing fit in?

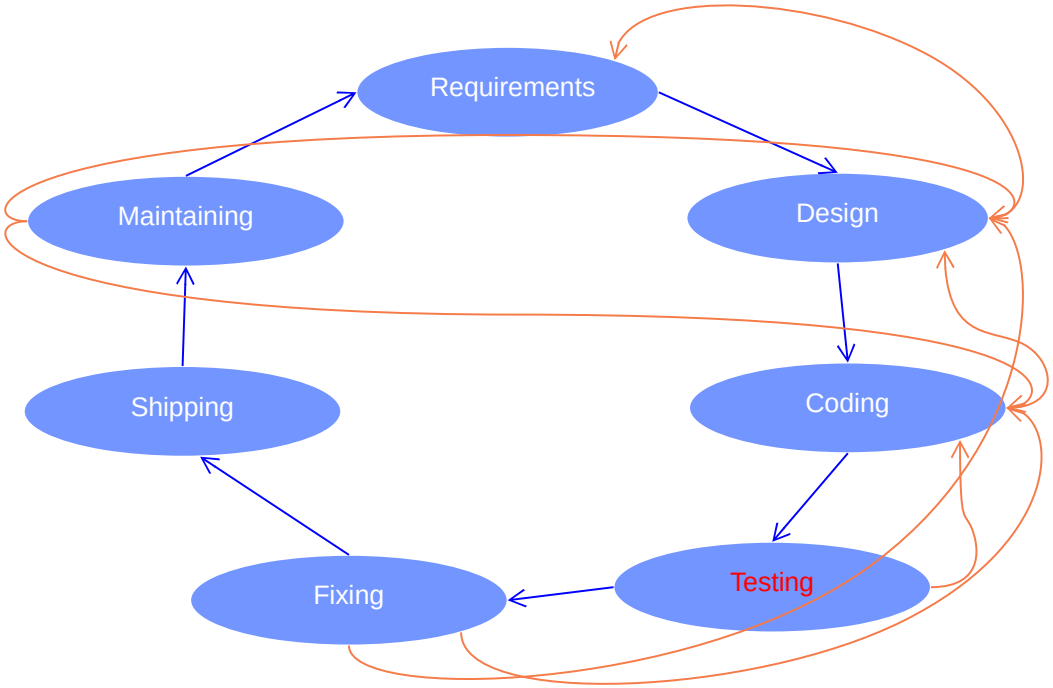
Development lifecycle: (very?) Classic view of testing (**testing the produced code post-compilation**)

- This type of testing has quite a complex interplay with other lifecycle activities – as can be seen.

- Traditionally the role of the software testers or the software test team / department.

- A complex and expensive activity in its own right.

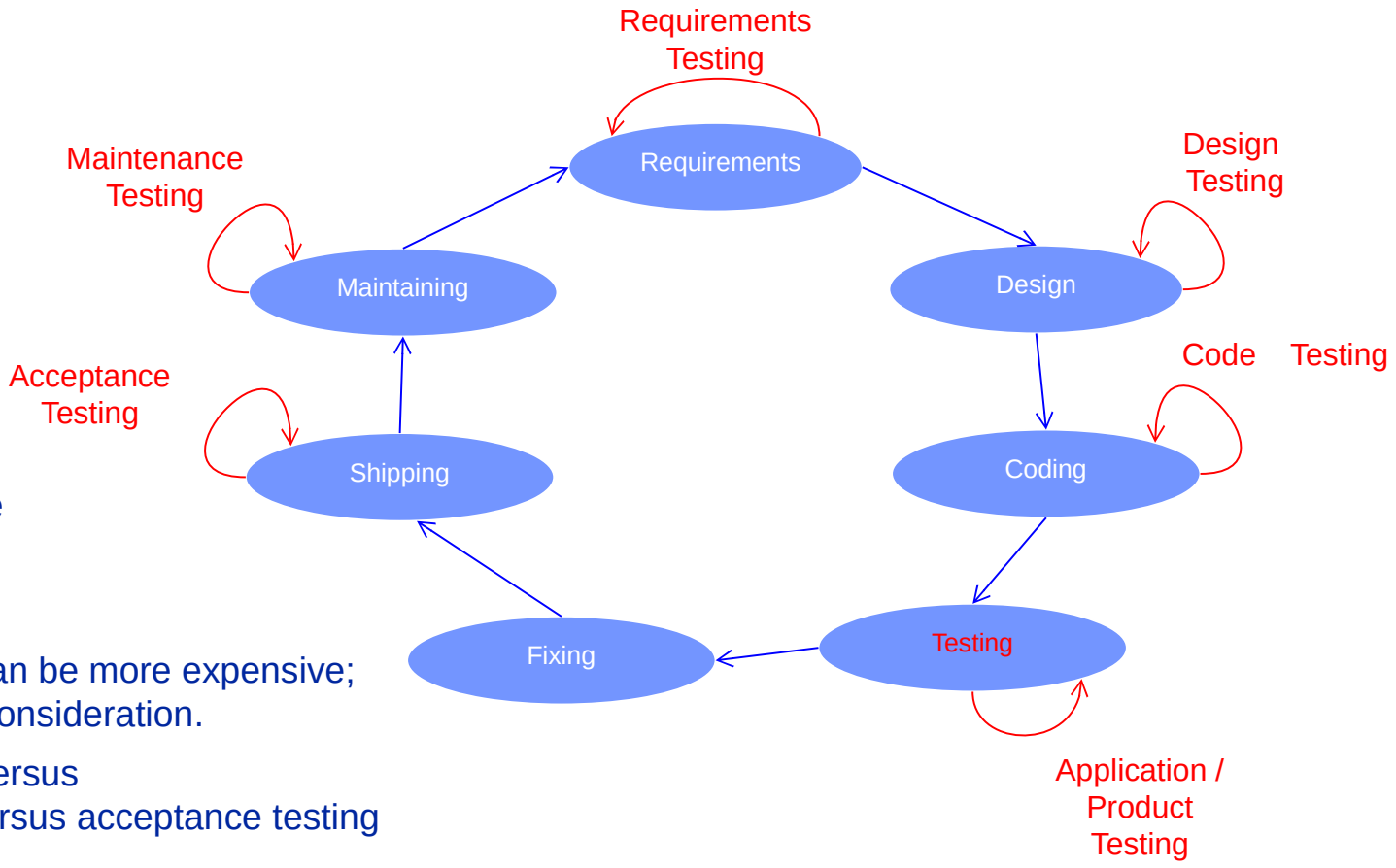
- Agile software development is different again.



- But this is just one aspect of the software testing domain.

How does testing fit in?

Development lifecycle: Broader scope of testing domain (**testing throughout the lifecycle**)



- Testing is everywhere

- Testing is expensive

- Not testing enough can be more expensive; there is an economic consideration.

- Note on integration versus system/qualification versus acceptance testing

- This module is concerned with all of these different types of testing.

Why is testing necessary?

Why testing is important?

After spending $130 million on it's problematical health insurance exchange, one of the 14 U.S. states that opted to create their own health insurance exchange (rather than utilize a federal-government-provided exchange) hired a new contractor in April 2014 to redo the site. Among the many problems with the initial site since it went live in October 2013, reportedly **hundreds of enrollees receiving enrollment information with names and birth dates of other enrollees**. It was estimated the **revamping would cost another $60 million**. Additionally, the primary contractor and subcontractor for the site are embroiled in a lawsuit with one another and at last report were in arbitration. Eventually it was reported that **the prime contractor agreed to pay $45 million to the government to avoid a lawsuit**.

7 out of 10 IT projects fail and the reasons for failure can often be attributed to poor software testing.

Software testing can represent c.40% of development costs within a project.

There are significant cost, time and quality benefits to getting software testing right.

Unfortunately, organisations often struggle to attain the desired balance between test coverage and time to delivery.

Certain test activities are one of the final activities.

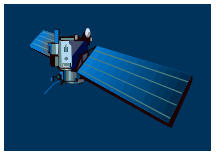
Because testing is one of the last tasks prior to customer delivery, well intentioned and appropriately planned testing activities are often subject to significant time pressure.

In this instance – it is not good to be last.

Software is a Skin that Surrounds Our Civilization

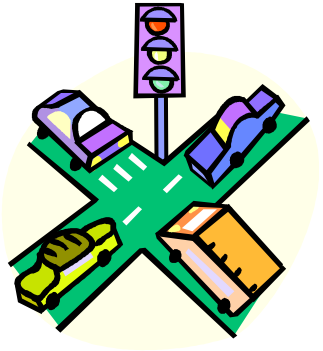


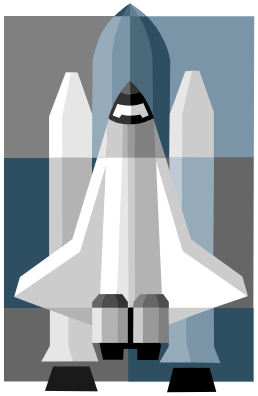










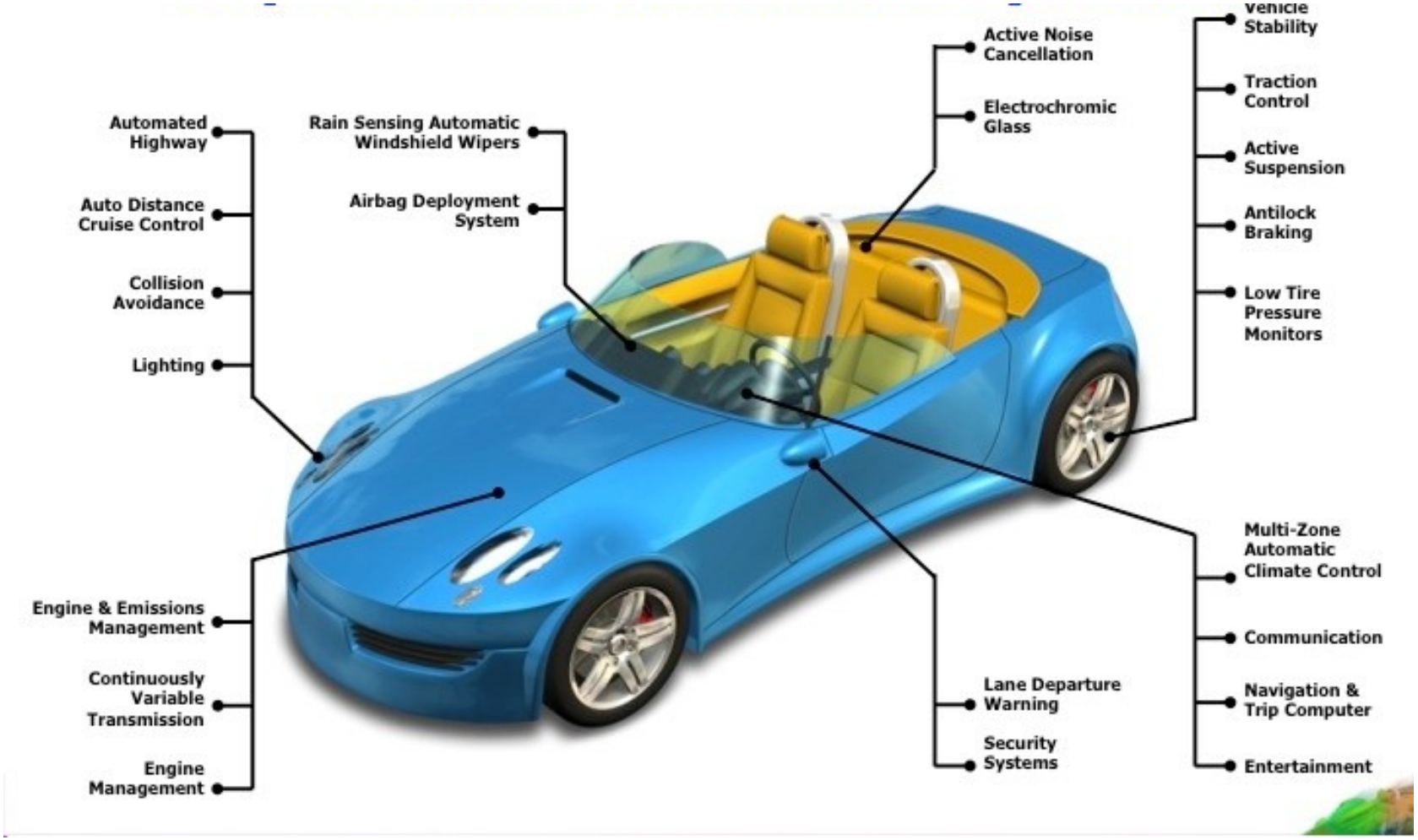



Trivia Quiz: Lines of code

>20 million

6.5 million

Think about what is really inside a car



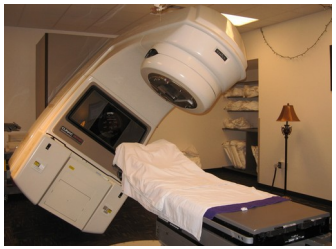
Failures in Production Software

- NASA's Mars lander, September 1999, crashed due to a unit integration fault—over $50 million US !
- Huge losses due to web application failures
  - Financial services : $6.5 million per hour
  - Credit card sales applications : $2.4 million per hour
- In Dec 2006, amazon.com's BOGO offer turned into a double discount
- 2007 : Symantec says that most security vulnerabilities are due to faulty software
- Stronger testing could solve most of these problems

World-wide monetary loss due to poor software is staggering

Types of failure

Examples of famous (or infamous) failures:



- Therac-25 (1985-1987)
- London Ambulance System (1992)
- Denver baggage handling system (1990's)
- Taurus (1993)
- Ariane 5 (1996)
- E-mail buffer overflow (1998)
- USS Yorktown (1998)
- Mars Climate Orbiter (September 23rd, 1999)

Why Does Testing Matter?

- NIST report, “The Economic Impacts of Inadequate Infrastructure for Software Testing”
  - Inadequate software testing costs the US alone between $22 and $59 billion annually
  - Better approaches could cut this amount in half
- Major failures: Ariane 5 explosion, Mars Polar Lander, Intel’s Pentium FDIV bug
- Insufficient testing of safety-critical software can cost lives:
  - THERAC-25 radiation machine: 3 dead
- We want our programs to be reliable
  - Testing is how, in most cases, we find out if they are


Ariane 5: exception-handling bug : forced self destruct on maiden flight (64-bit to 16-bit conversion: about 370 million $ lost)


Mars Polar Lander crash site?

Smaller scale – defects not publicised

- Software companies have to maintain their software, which is costly.
- Potential defects are reported – as issues - from users / clients
- Issues are prioritised, examined and resolved.
- Some issues are the result of defects.
- Triage is required.

Causes of software defects

A human being can make an error (mistake), which produces a defect (fault, bug) in the code, in software or a system, or in a document

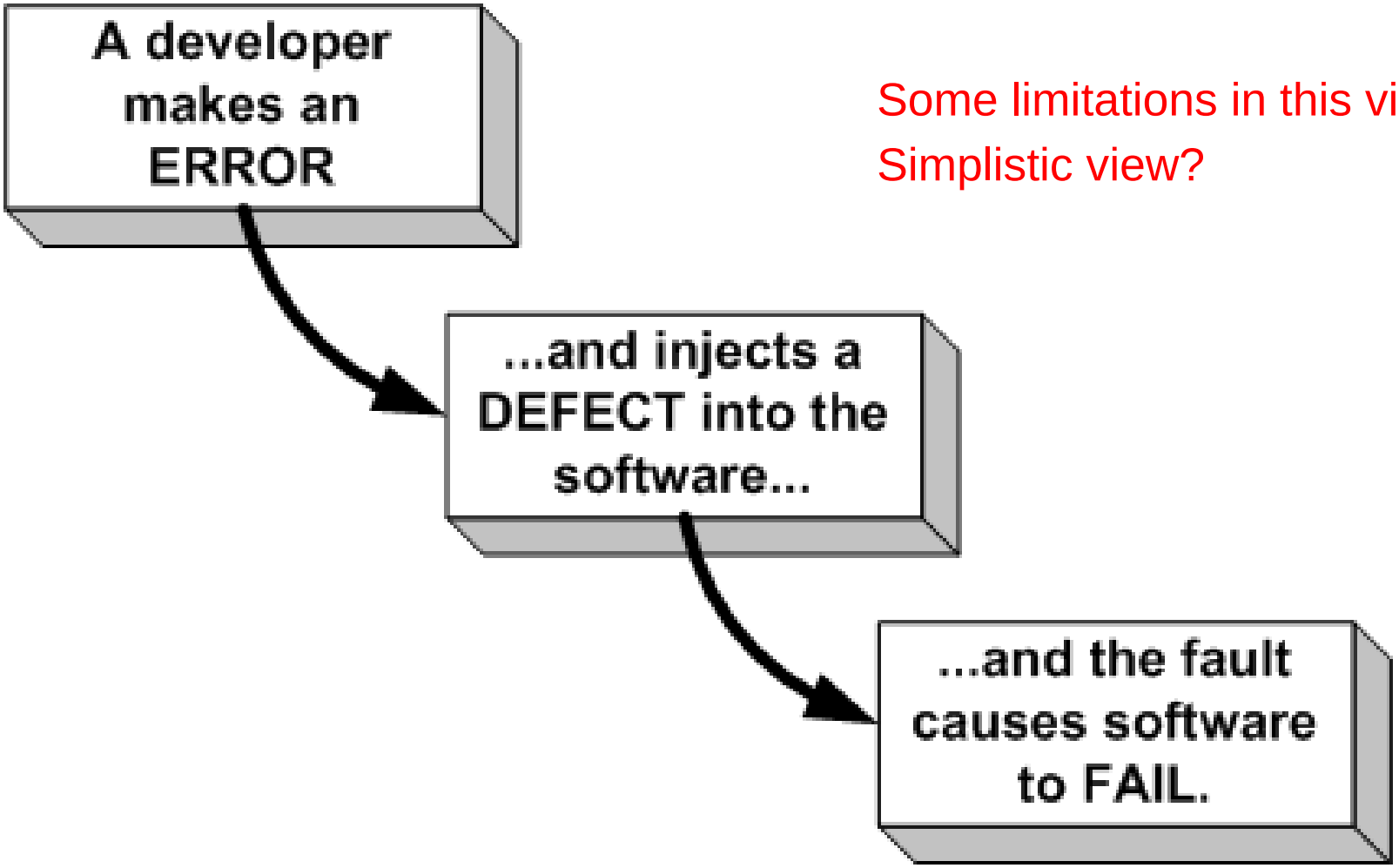
If the code is executed, the system will fail, causing a failure

Defects in software, systems or documents may result in failures, but not all defects do so

Failures can be caused by environmental conditions as well:

- radiation, magnetism, electronic fields, and pollution can cause faults in firmware or influence the execution of software by changing hardware conditions.

Errors, defects and failures



Some limitations in this view?

Simplistic view?

A failure is…

"A failure is present if a warrantable (user) expectation is not fulfilled adequately."

Examples of failures include: products that are too hard to use, or too slow, even if they still fulfill the functional requirements. i.e. when a user experiences a problem.

A failure is: a deviation of the software from its expected delivery or service

A failure occurs when software does the 'wrong' thing

Most of the time software does the right thing

Software defects cause software failures when the program is executed with a set of inputs that expose the defect.

Terminology…

Sometimes a failure will be called a “problem”, “incident”
“issue”

A defect is…

A manifestation of human error in software

Defects may be caused by requirements, design or coding errors

They are discovered either by inspecting code or by inferring their existence from software failures.

Terminology…

Sometimes a defect will be called a “bug” or a “fault”

An error is…

A human action producing an incorrect result

When programmers make errors they introduce faults to program code

Errors are not

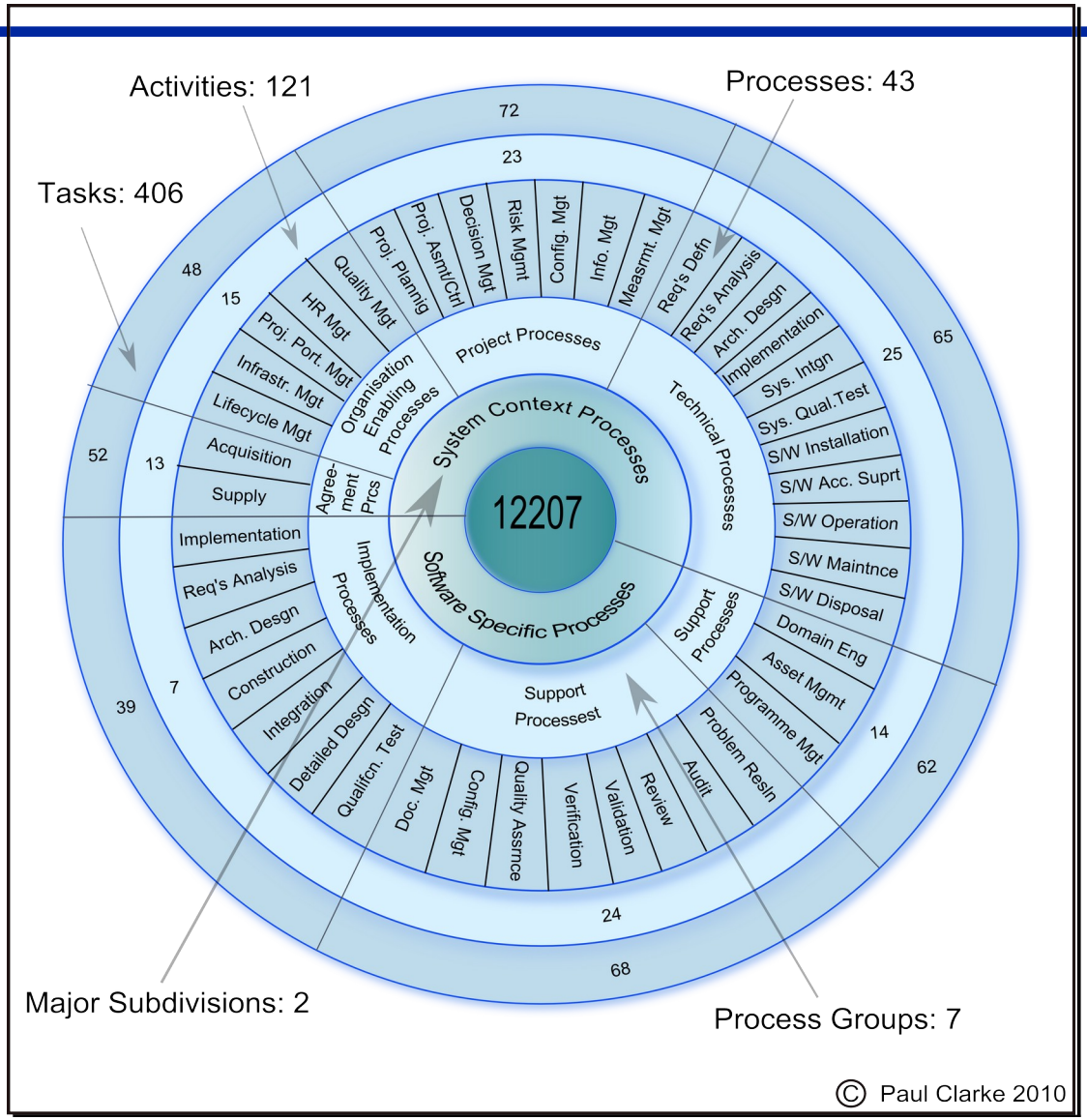
- Errors are not just accidents or mistakes
- Errors are not cured by "just being more careful"
- Errors are not necessarily an act of incompetence

Errors are inevitable in a complex activity.

Fundamentals continued…

- Software complexity
- Role of testing in development, maintenance and operations
- Testing and…..

How complex is software development?



Role of testing in development, maintenance and operations

Rigorous testing of systems and documentation can help to:

- reduce the risk of problems occurring in an operational environment
- and contribute to quality

Software testing may also be required:

- to meet contractual or legal requirements,
- or industry-specific standards.

Testing and confidence

If we buy a software package

- we may believe it works, but...
- a test gives us the confidence that it will work

When we buy a car, cooker, off-the-peg suit

- we assume they work, but do they work for me?
- we still test them

When we buy a kitchen, haircut, bespoke suit

- we were involved in the requirements
- and we test them.

Testing and contractual requirements

When we buy custom-built software, a contract will usually state

- the requirements for the software
- the price of the software
- the delivery schedule and acceptance process

We don't pay the supplier until we have received and acceptance tested the software

Acceptance tests help to determine whether the supplier has met the requirements.

Testing and other requirements

Software requirements may be imposed:

- laws governing our industry must be obeyed
- industry regulations must be adhered to
- our customers may insist we are compliant
- industry standards (e.g. safety-critical software)

When we write, change or buy software, we have to provide evidence that we comply

Testing provides most of that evidence.

Testing and quality

- Testing can measure quality of a product and indirectly, improve its quality.
- Testing measures quality
  - if we assess the rigour and number of tests and if we count the number of faults found...
  - we can make an objective assessment of the quality of the system under test
- Testing improves quality
  - tests aim to identify defects
  - if defects are found, we can improve the quality of the software.

Fundamentals continued…

- How much testing is enough?

How much testing is enough?

A perplexing question

There's no upper limit

If we don't test enough, we get blamed for faults in production

The tester can't 'win'.

Think of managing a deficit or a surplus.

How much testing is enough?

There are an infinite number of tests we could apply and software is never perfect

So how much testing is enough?

Objective coverage measures can be used:

- standards may impose a level of testing
- test design techniques give an objective target

But all too often, time is the limiting factor

- so we may have to rely on a consensus view to ensure we do at least the most important tests.

Complete Testing is not practically feasible (1)

Consider a simple little program that consists of a loop in which there are 5 possible execution paths from the start of the loop to the end of the loop.

If the maximum number of iterations of the loop is 20 then there is: $5^{20} + 5^{19} + 5^{18} + ... + 5^{1}$ different execution traces. That's about 100 quadrillion different execution traces.

If each test is done manually and takes 5 minutes to test each trace then the full test takes 1 billion years.

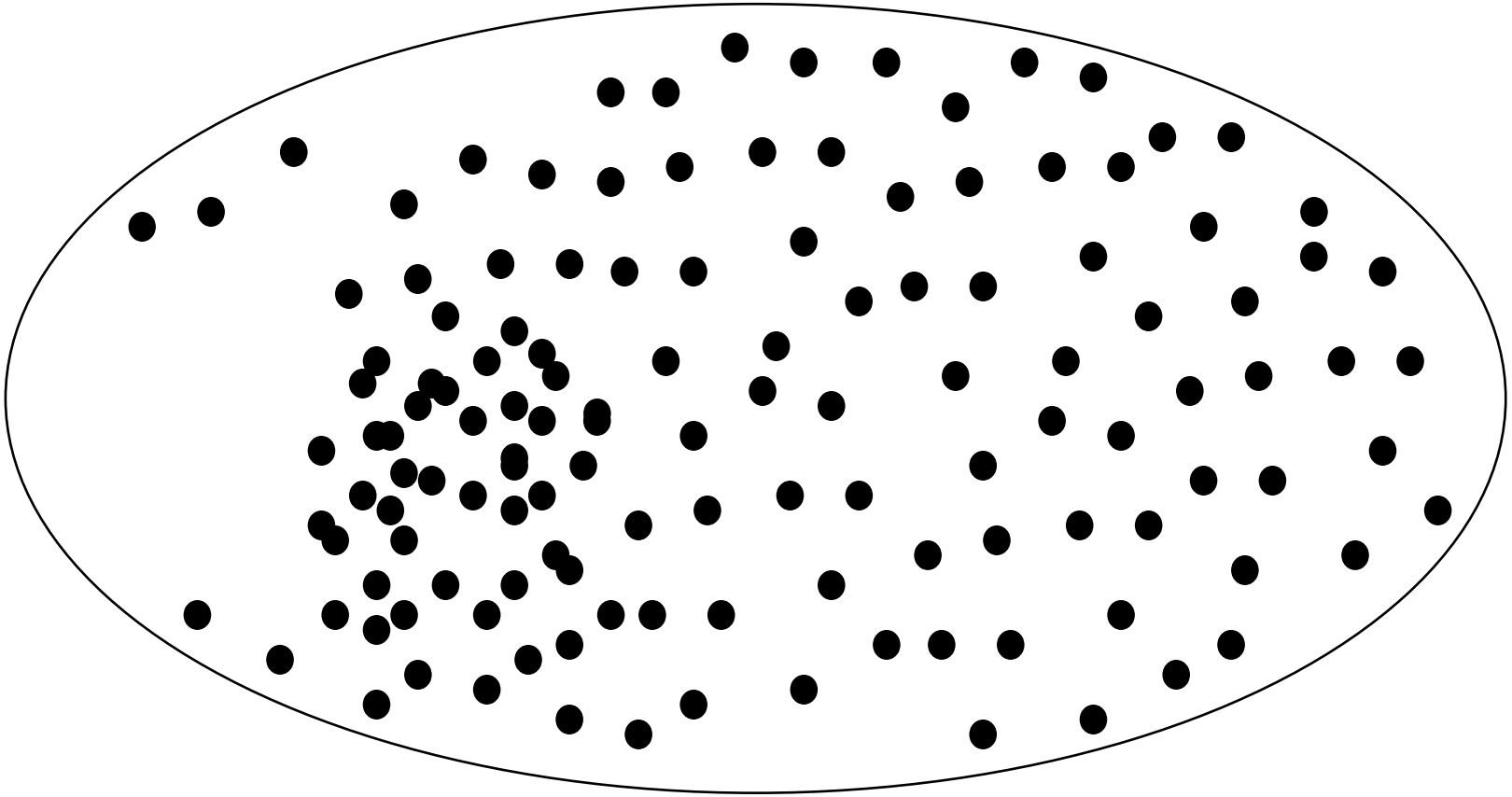
If automated so that each test takes 5 microseconds, then it would take 19 years!

Complete Testing is not practically feasible (2)

Complete testing is nearly impossible because of the following reasons:

- The domain of possible inputs of a program is too large to be completely used in testing a system. There are both valid inputs and invalid inputs.
- The program may have a large number of states.
- The design issues may be too complex to completely test (especially for larger systems)
- It may not be possible to create all possible execution environments of the system

The bugs that lurk in our systems



Fundamentals continued…

- Where are the bugs?

Where are the bugs?

Of course, if we knew that, we could fix them and go home!

Experience tells us

- bugs are sociable! - they tend to cluster
- some parts of the system will be relatively bug-free (bought-in or unchanged code)
- bug fixing and maintenance are error-prone – and some changes cause unintended faults.

Testing as a fishing net

Size of the net reflects size of the test

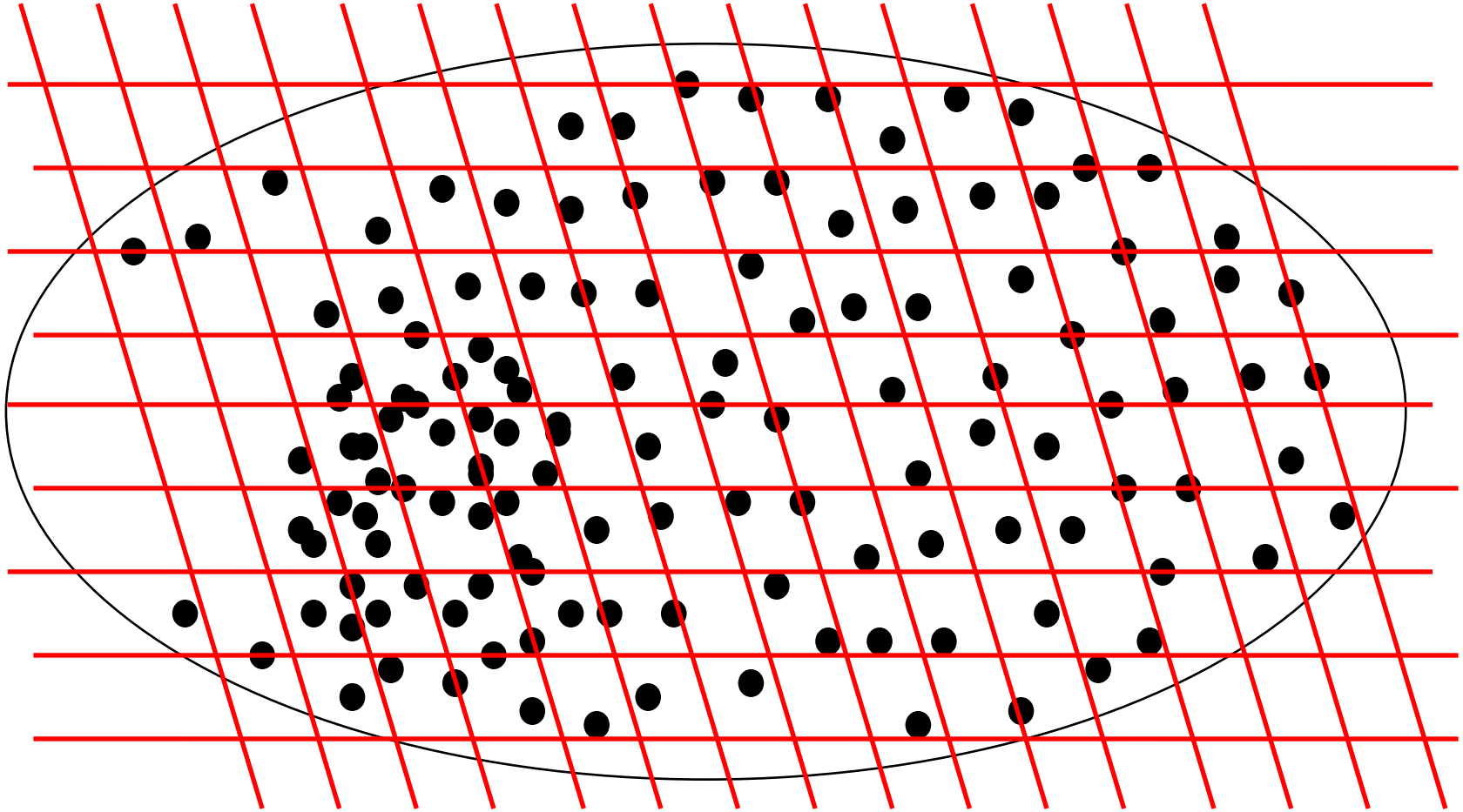
Density of the mesh reflects test depth or thoroughness

Large nets, large mesh to give overall confidence

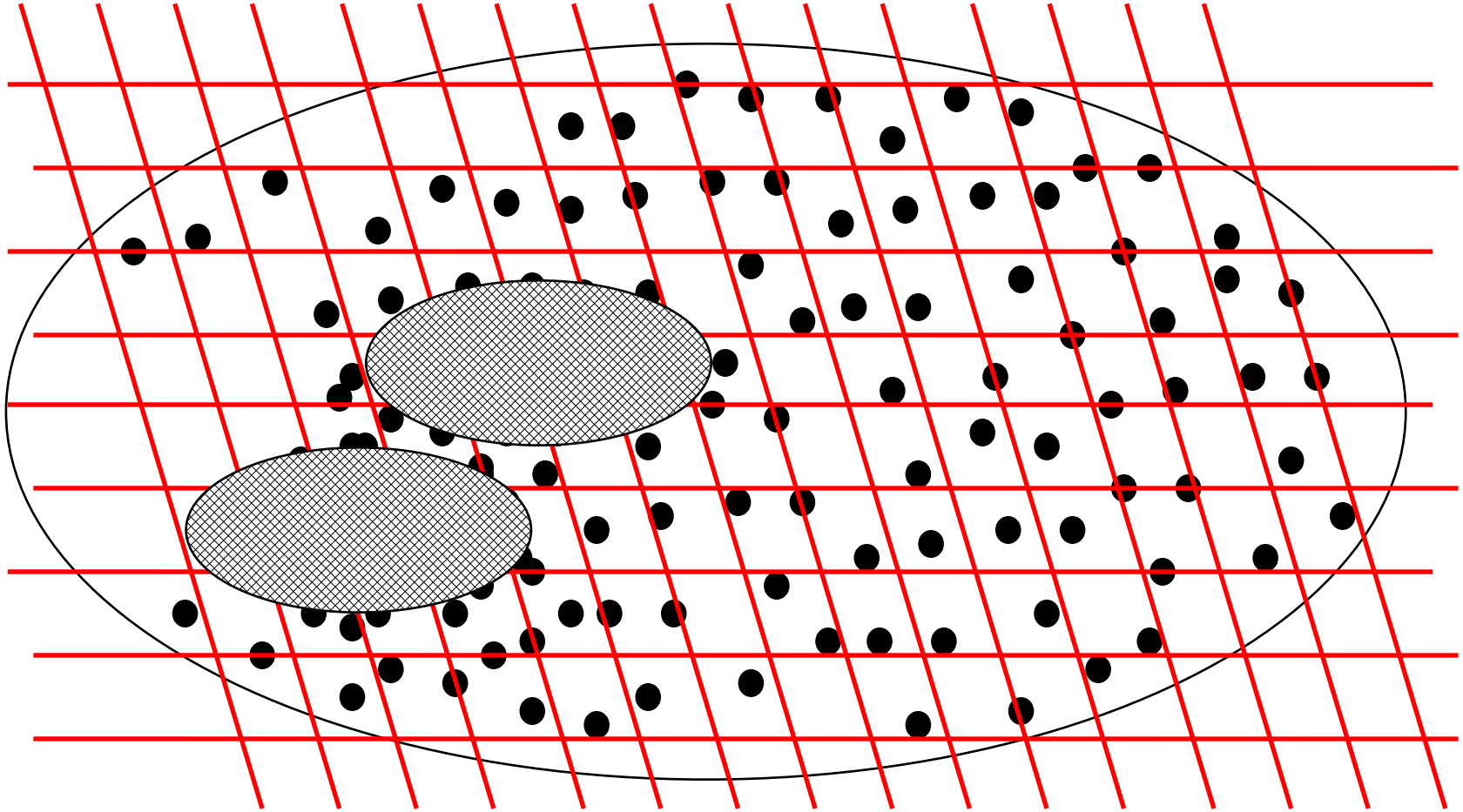
Small nets, fine mesh to find bugs

- use in areas that are critical to make sure the bugs aren’t there
- use in areas where we expect lots of bugs.

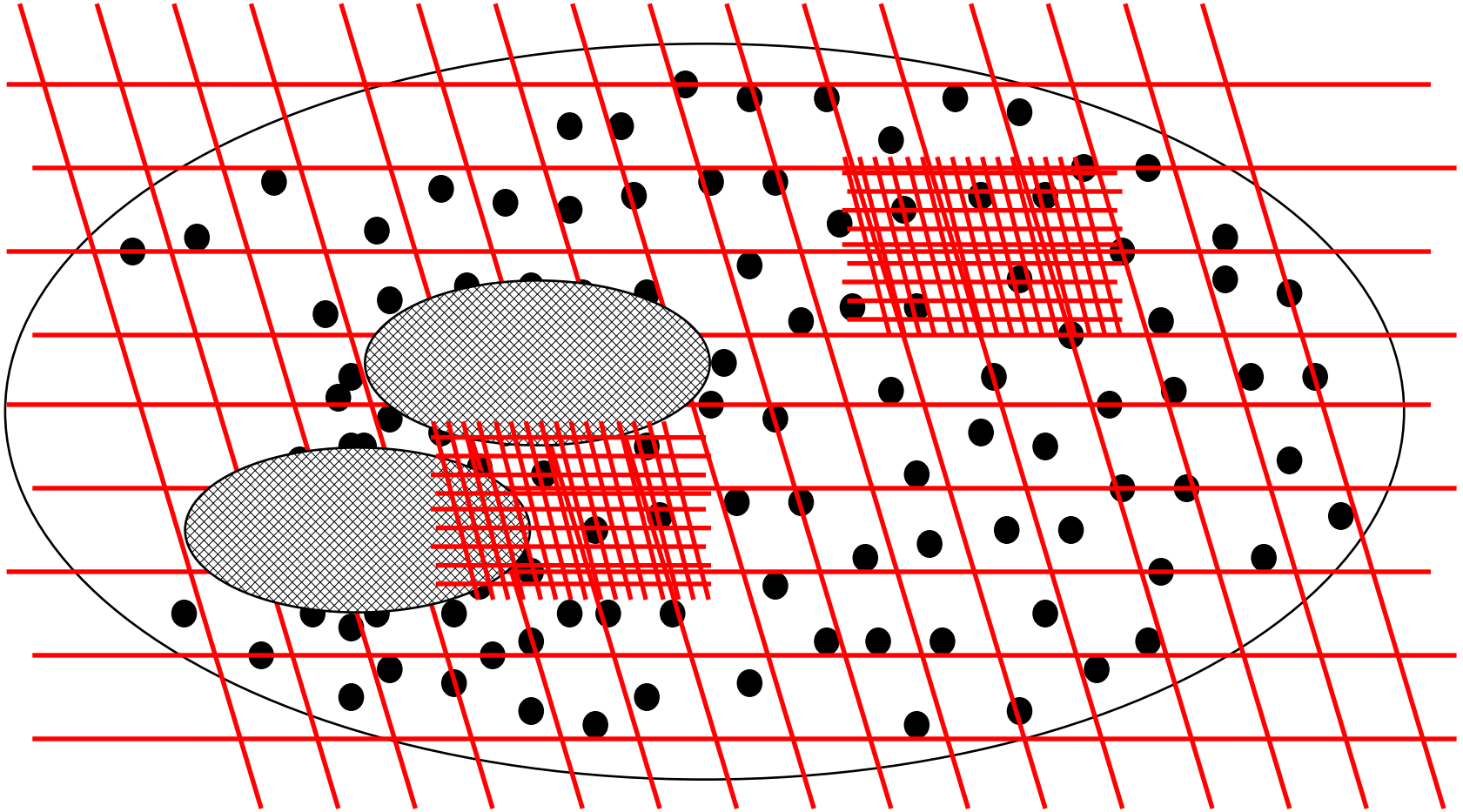
Coverage of business-oriented tests



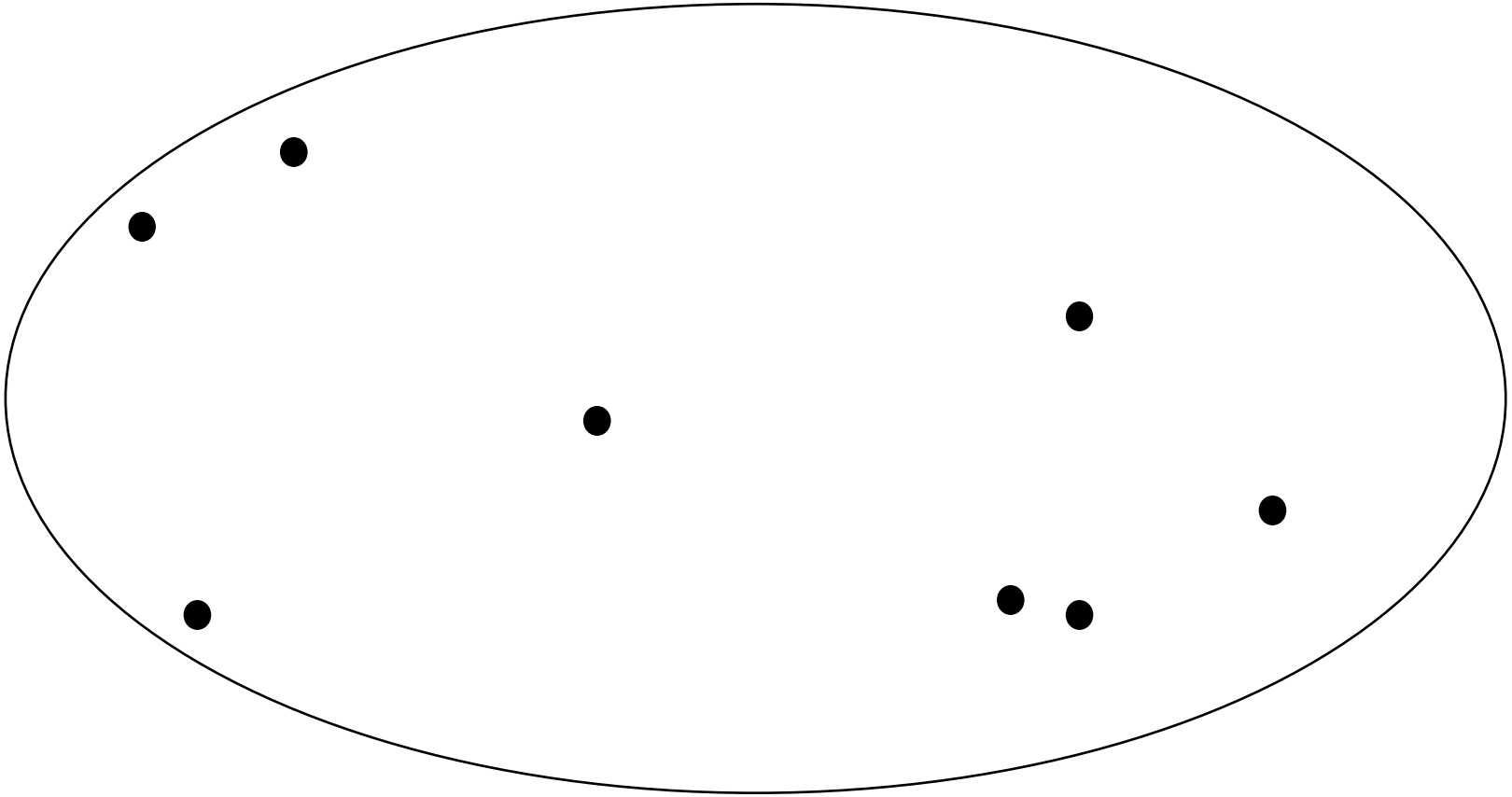
Rigorous tests of the riskiest parts of the system



More tests of the business critical parts of the system



What about the bugs we don’t find?



What about the bugs we don’t find?

- If not in the business critical parts of the system - would the users care?
- If not in the system critical parts of the system - should be low impact
- If they are in the critical parts of the system
  - should be few and far between
  - should be pushed to the domain of obscure usage.

Fundamentals continued…

- What is testing?

What is testing?

Testing

The systematic exploration of a component/system with the main aim
of finding and reporting defects.

Testing rigorously examines the component/system behaviour and reports defects found.

Tests are repeated to ensure that defect corrections have been effective.

Debugging

A process undertaken by developers

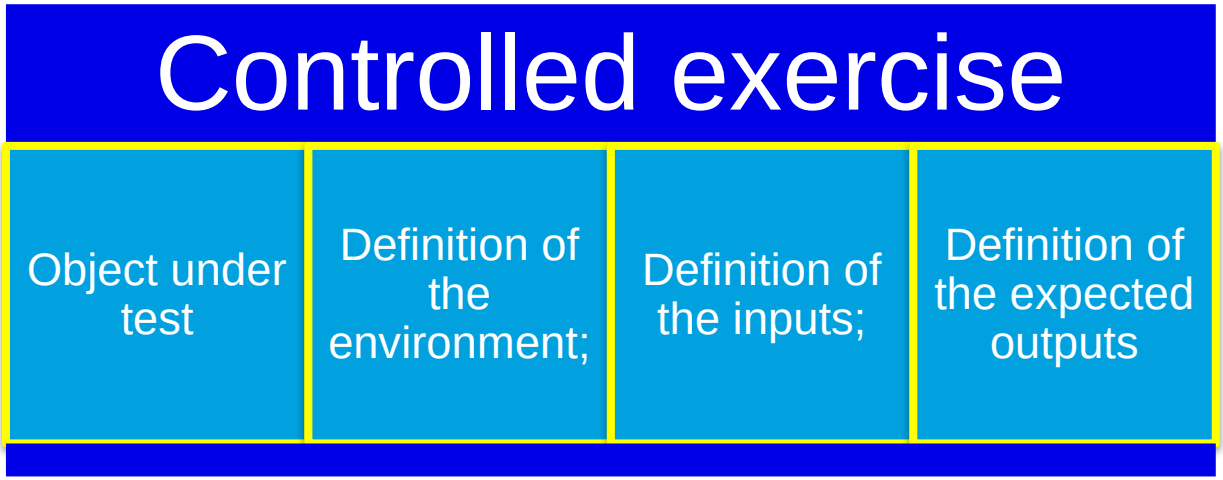
to identify the cause of defects in code and undertake corrections.

Debugging is done first to ensure that the component or system is at a level
to enable rigorous testing.

Debugging can be used to understand the root cause of observed failures.

What is testing?

- What is a Test?



- Thus once a Test is run, you'll get
  - An actual result / output
  - A decision on its success

Testing isn't just running tests

planning and control

choosing test conditions

designing test cases

preparing expected results

checking actual results against expected results

evaluating completion criteria

reporting on the testing process and system under test

and closing the test phase.

Test Objectives

There can be different test objectives:

- finding defects
- gaining confidence about the level of quality and providing information
- preventing defects

In acceptance testing, the main objective may be to confirm that the system works as expected, to gain confidence that it has met the requirements.

Fundamentals continued…

- Static and Dynamic Testing

Static and Dynamic Testing

Static Testing

Testing without executing the program

- This include software inspections and some forms of analyses
- Very effective at finding certain kinds of problems – especially “potential” faults, that is, problems that could lead to faults when the program is modified

Dynamic Testing

Testing by executing the program with real inputs

- Includes various techniques but requires the coding stage to be completed.

Dynamic and Static Testing

Testing also includes reviewing of documents (including source code).

Both dynamic testing and static testing can be used as a means for achieving similar objectives, and will provide information in order to improve both the system

Reviews of documents (e.g. requirements) also help to prevent defects appearing in the code.

Static testing in the lifecycle

Static tests are tests that do not involve executing software

Reviews, walkthroughs, inspections of (primarily) documentation

Requirements

Designs

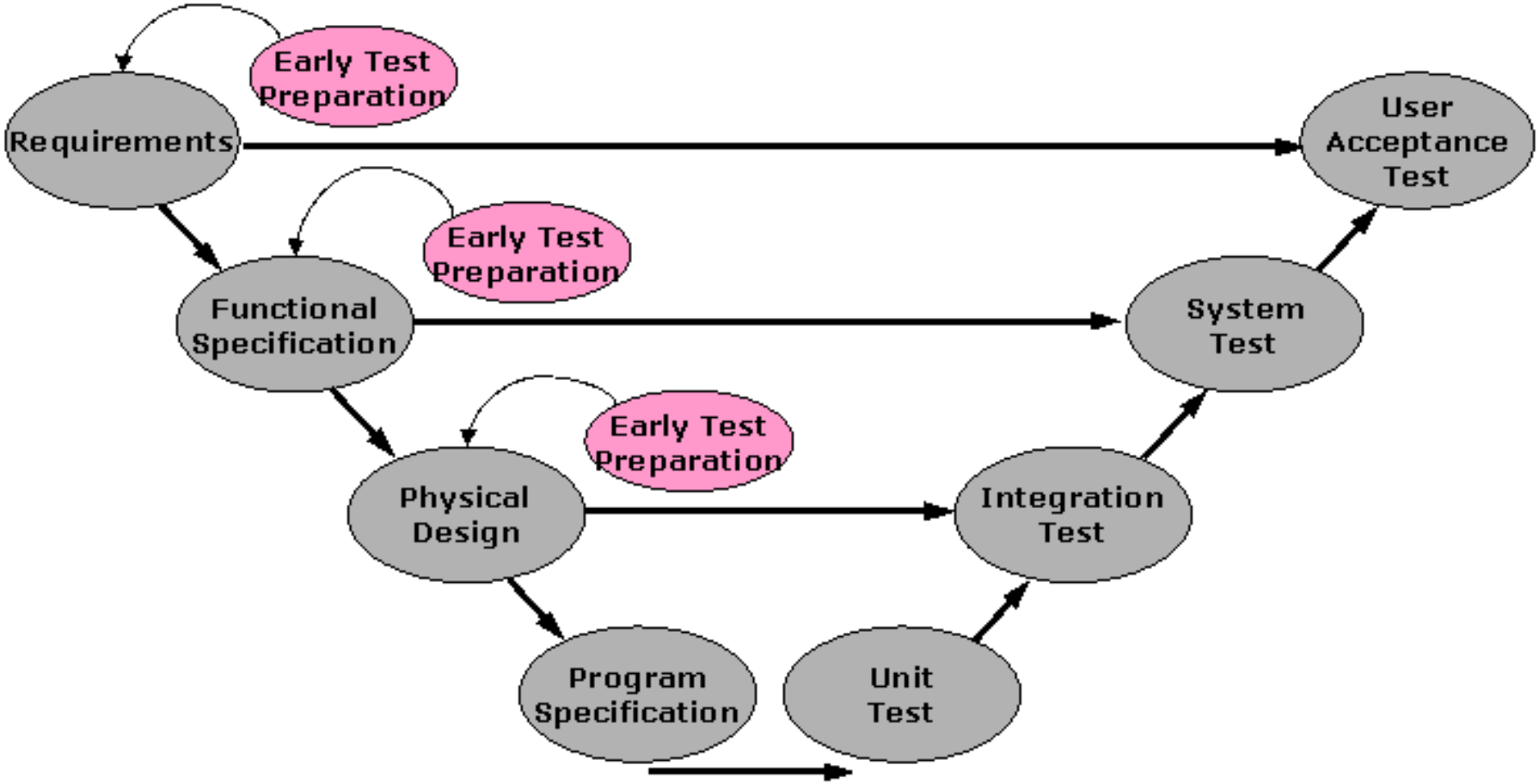
Code

Test plans.

Dynamic testing in the lifecycle

- Program (unit, component)
- Integration or Component Integration testing
- System testing
  - non-functional (e.g. performance, security, backup and recovery, stress)
  - functional where functionality as a whole is evaluated
- User acceptance testing.

Early test case preparation



Fundamentals continued…

- Types of Testing

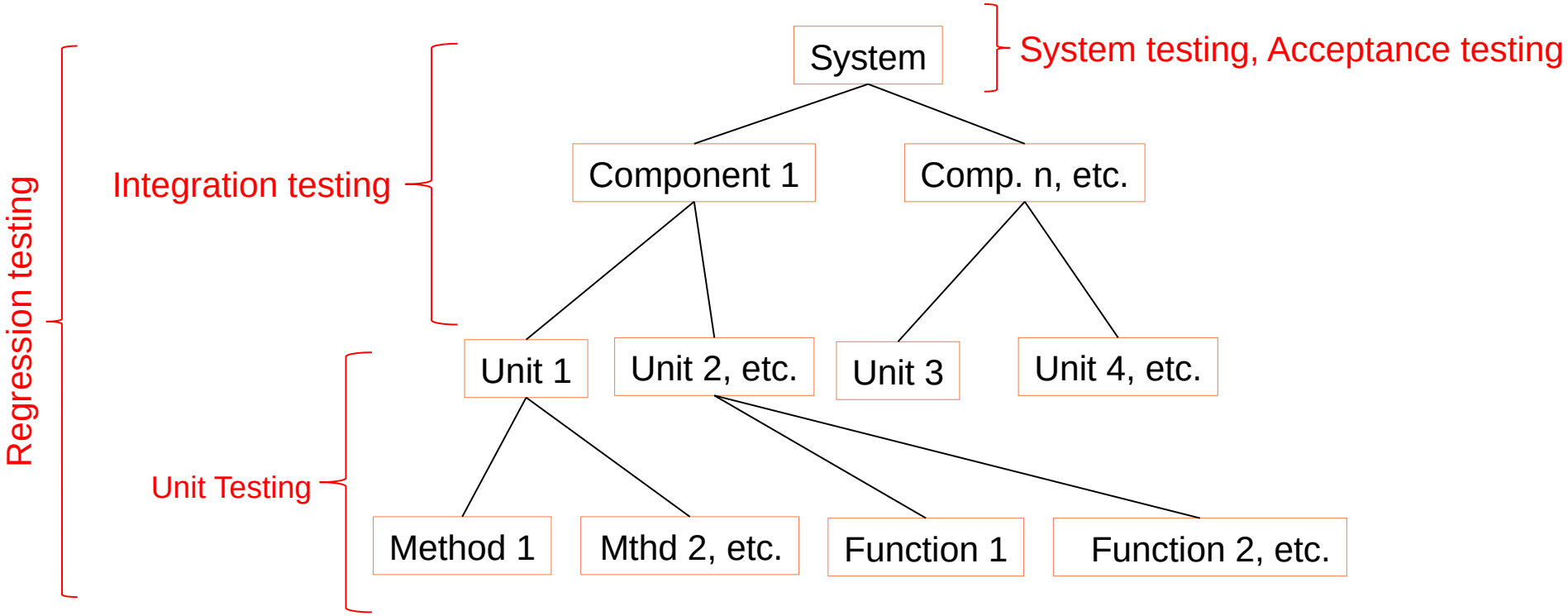
Types of Testing

In some cases the main objective of testing may be to assess the quality of the software (with no intention of fixing defects), to give information to stakeholders

Regression/Maintenance testing often includes testing that no new errors have been introduced during development of the changes.

During operational testing, the main objective may be to assess system characteristics such as reliability or availability.

Types of Dynamic Testing



Each Type Of Testing

Each type of testing addresses a specific concern:

Unit Testing: addresses the quality of individual units (e.g. methods, functions)

Integration Testing: checks if individual components (which are a combination of two or more individual units) operate together

System Testing: checks if the system as a whole delivers the desired functionality.

Other types of testing also exist that address other concerns, e.g. acceptance testing allows users or a designated user representative to establish if the system is acceptable to the users.

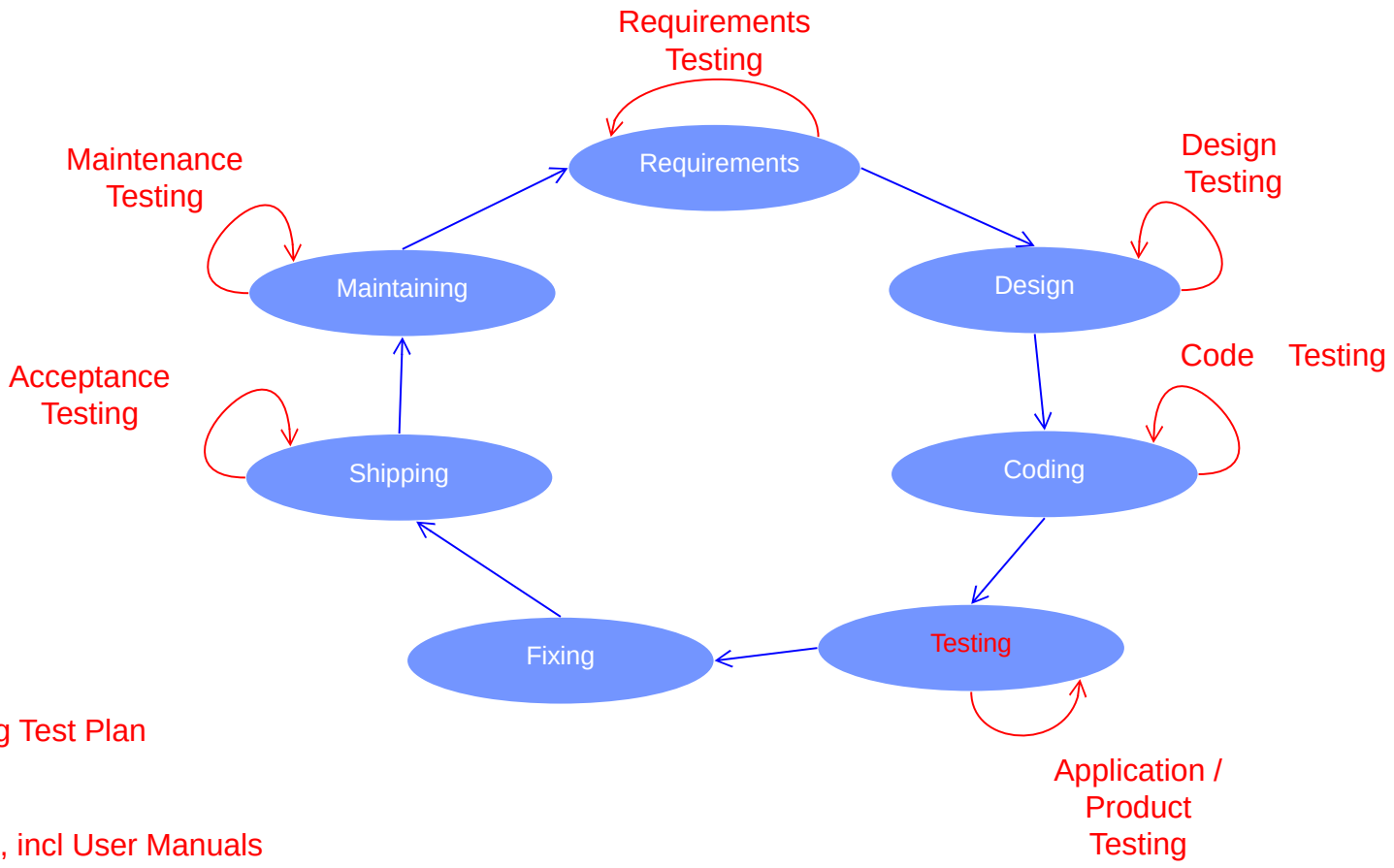
Types of Static Testing

Mostly taking the form of:

Reviewing

Inspecting

Software Lifecycle



Other testing:

Testing plans, including Test Plan

Testing test cases

Testing documentation, incl User Manuals

Remember: Debugging and Testing

- Debugging and testing are different
- Testing can show failures that are caused by defects
- Debugging is the programmer activity that identifies the cause of a defect, repairs the code and checks that the defect has been fixed correctly
- Subsequent confirmation testing by a tester ensures that the fix does indeed resolve the failure
- The responsibility for each activity is very different, i.e. testers test and developers debug.

Fundamentals continued…

- End of Week 2 Materials